



Odin

An Autonomous cloud infrastructure management system for Monitoring, analysing, and responding to operational events ensuring reliability, scalability and resilience across multiple cloud instances.

Supervisor: David Drohan

HDip in Computer Science

Michael O Driscoll – 20099693

Contents

Background	3
Introduction	3
Research Analysis Modelling and Planning (RAMP)	4
Research / Analysis	4
User Requirements	4
Feature Analysis	5
Market Analysis.....	6
Repository Analysis.....	7
Modelling.....	8
Data Model.....	8
Pipeline Model	9
System Model	10
Planning.....	11
Methodology	12
Technical Implementation	13
Charting metrics collected on a Database	13
Automated Action on collected Metric Data	15
AI-Driven comments and System Administrator Control	16
Running Python Scripts from JavaScript	19
Monitoring multiple Instances	19
Conclusion.....	20
References	21
Appendices	24
LLM Prompts	24

Figure 1 - Nagios IT Monitoring Solution	6
Figure 2 – Zabbix IT Monitoring Solution	6
Figure 3 - Uptime Dashboard Example	7
Figure 4 - Pipeline Model from data source through to dashboard	9
Figure 5 - System model showing flow of data and actions between system parts	10
Figure 6 - Initial Gannt Chart	11
Figure 7 – Iterative and Incremental Development [40]	12
Figure 8 - First attempt at storing data in MongoDB via Python	13
Figure 9 - Use of handlebars syntax to present data in chart	13
Figure 10 – Displayed result from figure 8	14
Figure 11 - Reload page every 30 seconds	14
Figure 12 - Initial python collector script	14
Figure 13 - Rule Based approach to performing actions on data.....	15
Figure 14 - Further additions to figure 12	15
Figure 15 - Dashboard	16
Figure 16 - Automated opinion from Odin.....	17
Figure 17 - Opinion Log.....	18
Figure 18 – Memory Usage and Database Storage Usage presented	18

Background

As a System Administrator in the pharmaceutical industry, it has been observed that a substantial amount of time is dedicated to system admin checks to monitor system health. This project addresses this issue by reducing the time spent on system admin checks.

Introduction

Odin addresses the issue by surveying a cloud system. This action can be done using an AI or rule-based control layer (another cloud system) that can monitor metrics, perform required actions and log these actions and metrics in a database, which can be accessed by a system admin dashboard or frontend application to present important information about the system health. This significantly reduces the time system admins perform system admin checks.

Research Analysis Modelling and Planning (RAMP)

Research / Analysis

User Requirements

Full-Stack VPC

- A Virtual Private Cloud containing servers and databases.
- Database shall switch over to a backup database in the event of database failure.
- Web page called “Venuely” for public user access that collects test user name and password and stores it securely on a database, allows users to Add Venues and associated events.
- Admin access available on the web page.

AI or Rule-based VPC

- A Virtual Private Cloud containing servers and databases.
- Database shall switch over to a backup database in the event of database failure.
- Server instances shall contain AI or Rule-Based scripts that communicate with servers and/or databases in Full-Stack VPC.
- Scripts shall collect data and store them in local databases.

Admin Dashboard

- The Admin Dashboard may be installed in a server within the AI or Rule-Based VPC or on a separate server.
- Admin Dashboard shall contain metrics, graphs, and any information considered important or relevant.
- Admin Dashboard may enable a user to execute actions (system administrator) that the AI or Rule-based script failed to execute or required a decision from the system administrator.

Feature Analysis

Full-Stack VPC

- Virtual Private Cloud (Full Stack VPC) – refer to [2]
- EC2 Image with web application installed from GitHub. [1]
- EC2 Image with MongoDB Installed
- >3 EC2 Instances
- 3 EC2 instances (MongoDB instances) [3] [5]

AI or Rule-based VPC

- Boto3 script installed on EC2 instance [4]
- EC2 image with installed scripts
- >2 EC2 Instances
- 3 EC2 instances (MongoDB instances) [3] [5]

Admin Dashboard

- Create Dashboard with the following resources [6]

Market Analysis

- Nagios Core Is a monitoring solution that ensures system health and performance. The Dashboard that will be created for this project will operate in a similar fashion, and there are instant alerts to detect issues [7] [8].

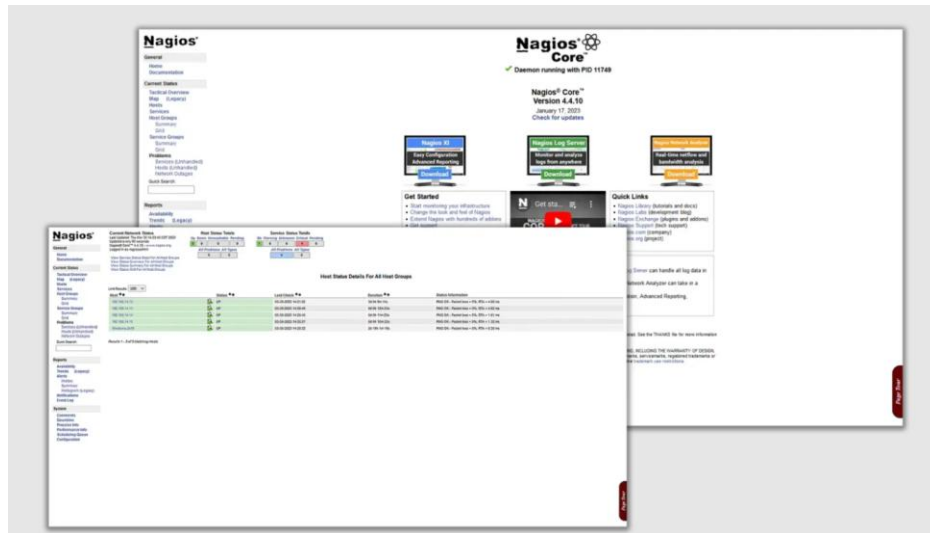


Figure 1 - Nagios IT Monitoring Solution

- Zabbix collects metrics from servers, virtual machines and websites and can display all information on one dashboard, this is a very similar solution to the dashboard that will be created [9].

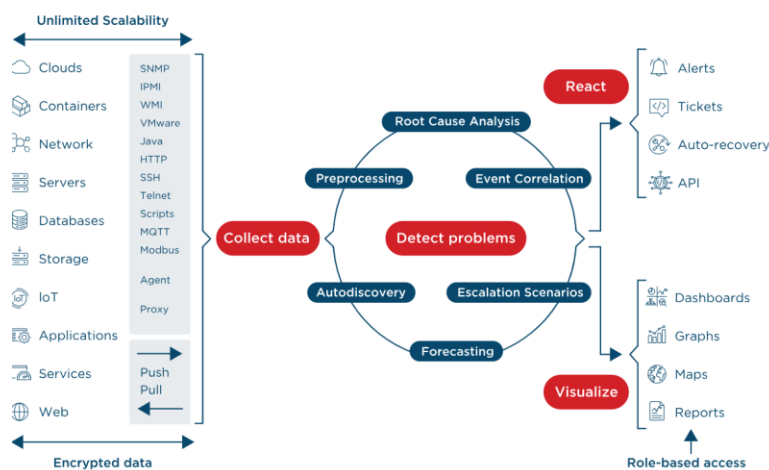


Figure 2 – Zabbix IT Monitoring Solution

Repository Analysis

AI or Rule-based VPC

Agentic DevOps

A repository was found that has a similar approach to the intended execution of this project. It uses AI to make decisions that self-heals the systems by detecting issues before they have an impact. It removes repetitive tasks using autonomous automation and provides vulnerability detection for security purposes [10].

This repository uses OpenAI's Agents SDK, to define an Agent, which solves tasks, and may be dedicated to a specific purpose, which uses specific tools [11]. The repository's architecture section uses an "EC2 Agent" which is useful in this project.

Admin Dashboard

Uptime

This repository uses Node.js which would inspire how a dashboard may look in this project, it uses MongoDB as it's database which is what is intended for this project. The image below (Figure 4) shows downtime, response times, and includes a graph of availability, parts of this repository may prove useful for this project [12].

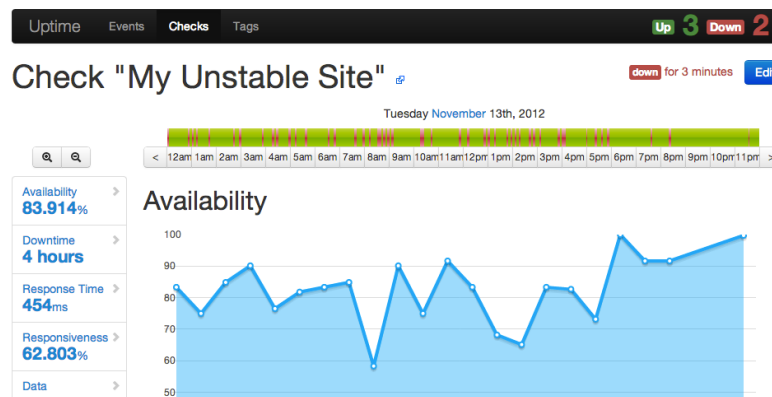


Figure 3 - Uptime Dashboard Example

Lunalytics

Lunalytics repository was discovered and contains some graphical elements that could be used for the dashboard in this project. It uses Node.js & React, but does not have MongoDB support, perhaps MongoDB support could be contributed to the repository through this project [13].

Modelling

Data Model

InstanceStats collection

The data model would consist of the following:

- Timestamp (Date and time of measurement)
- CPU (Central processing unit)
- memUsed (Used Memory Percentage)
- uptime (Accumulated time since last startup)
- storUsed (Percentage of database storage used)
- Bandwidth (bits per second)

Another type would be logging an action from the AI/Rule-Based script:

- Time stamp (Date and time of action)
- Action (What action was taken on the system)

This will also be stored in its own separate table.

Pipeline Model

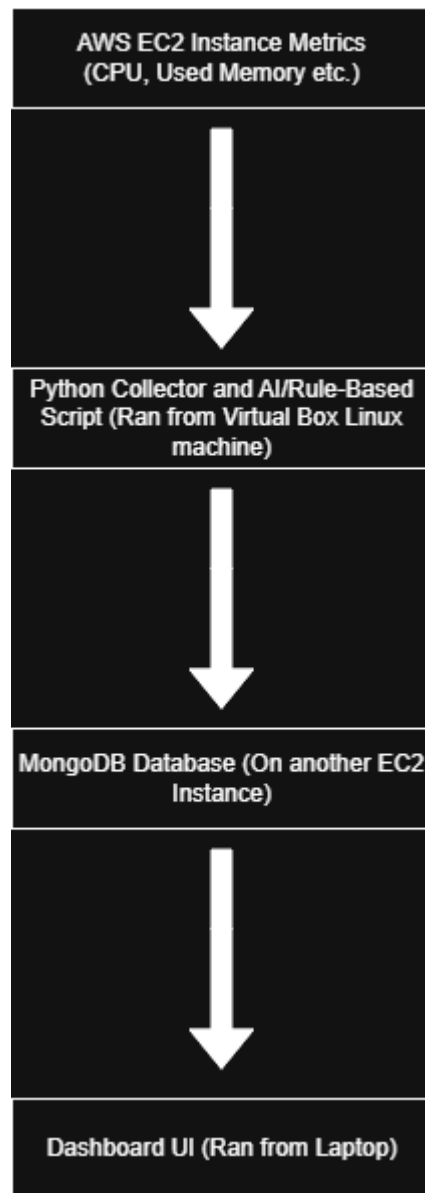


Figure 4 - Pipeline Model from data source through to dashboard

System Model

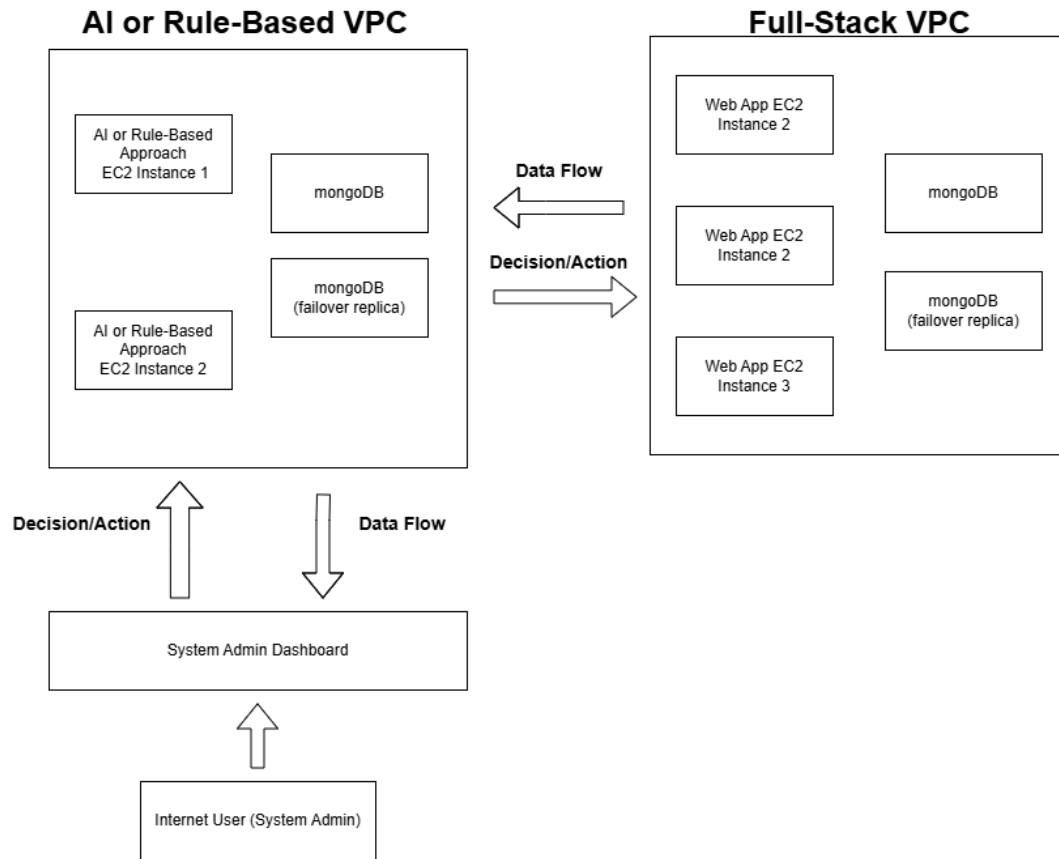


Figure 5 - System model showing flow of data and actions between system parts

Planning

Initial plan through Gantt chart:

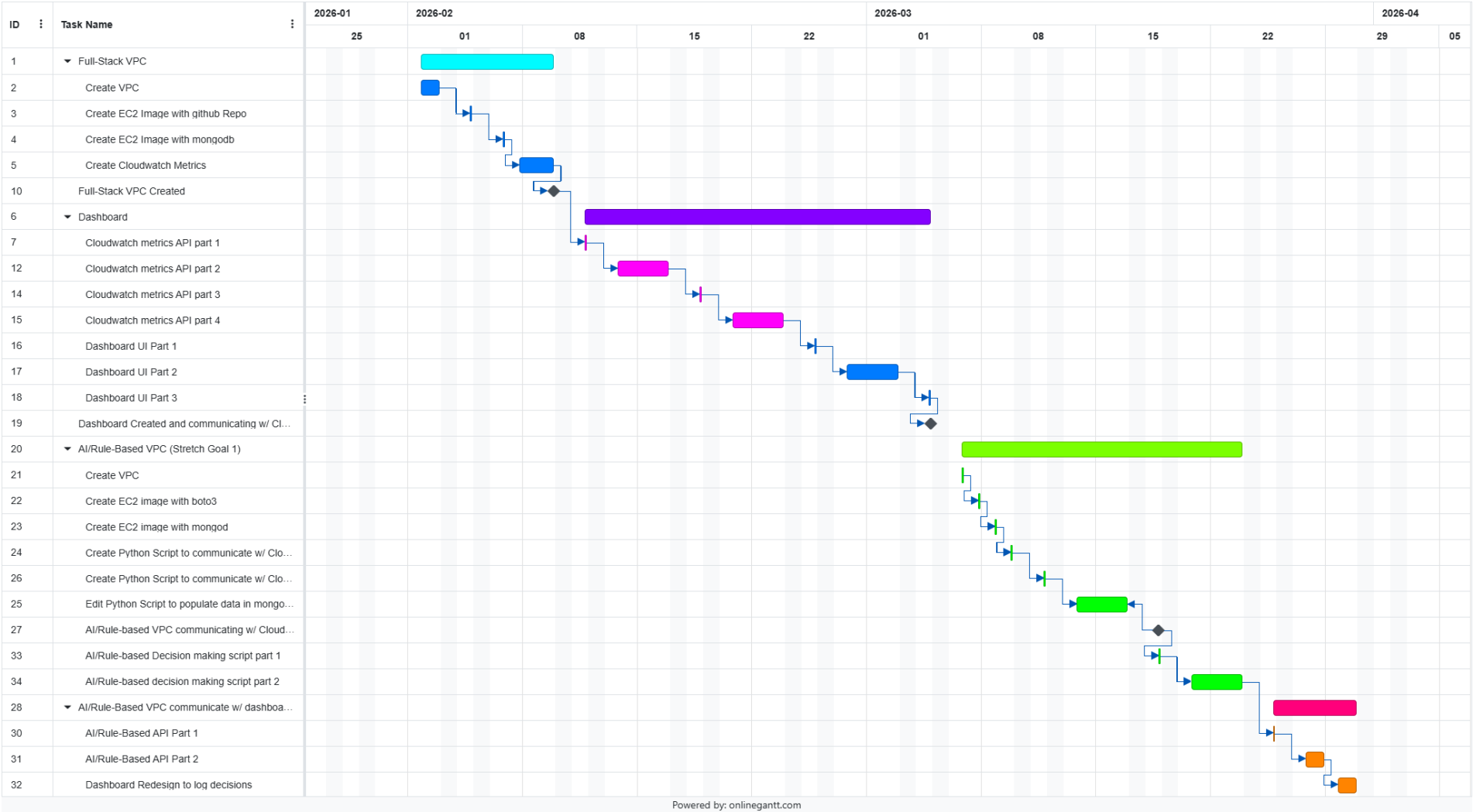


Figure 6 - Initial Gantt Chart

Methodology

“Incremental Development” was used to implement the project, and how other smaller projects and assignments have been implemented in the past.

Where by a small part of the code was written then tested to see if it would function properly. E.g. “Graph was coded, does it plot the points?, if so, move onto the next feature, if not, recode the graph”

A commit and push to GitHub would not be completed until a feature is working.

Unless, there was a feature that could not be fixed at that time, would then have been pushed to github with a comment explaining so.

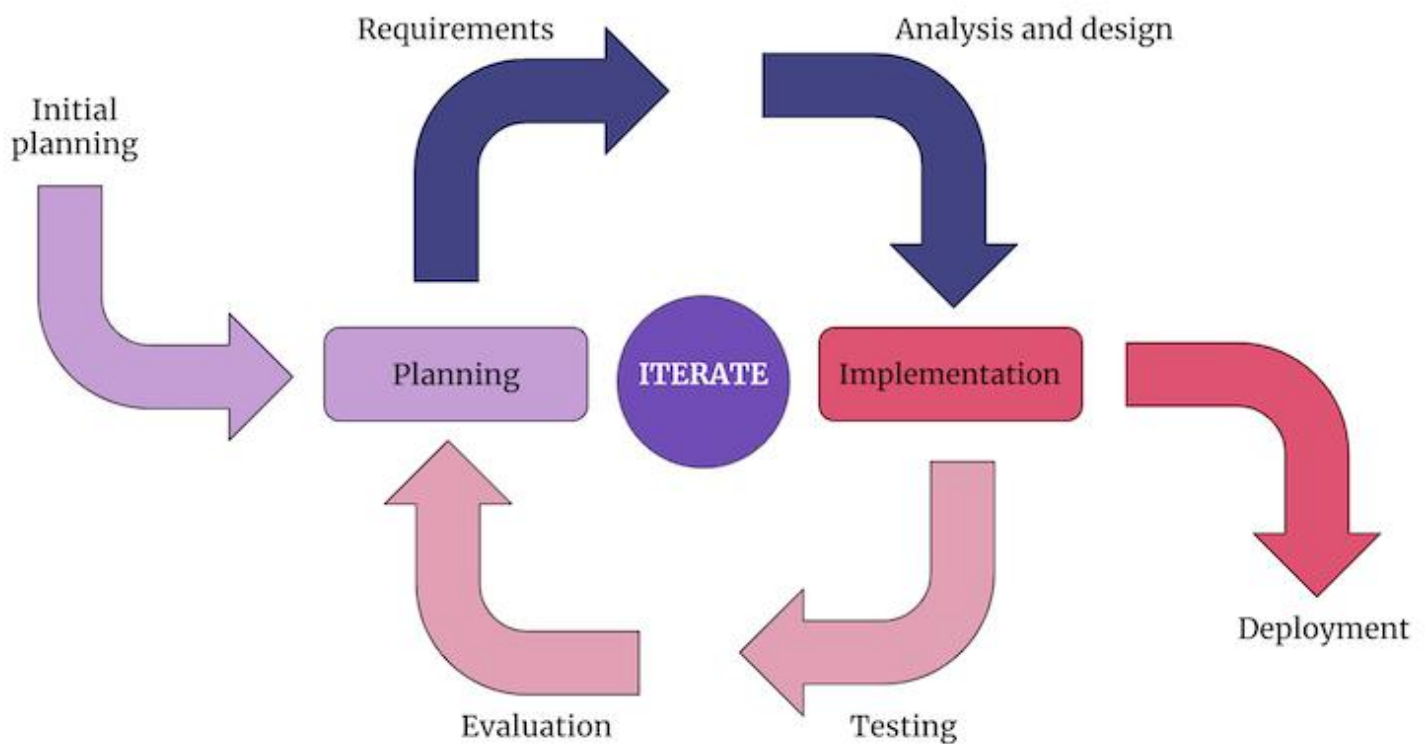


Figure 7 – Iterative and Incremental Development [40]

Technical Implementation

The project implementation was started by cloning an old assignment from Full-Stack Development in Semester 3 [1]. This project was renamed and a new repository was created under the name of Odin [17]. Immediately adjusted the page style to suit this project, the colour “Blue” was chosen to give the user a sense of “trust” as learned from a previous module [14].

Charting metrics collected on a Database

A Basic introductory bar chart was added to the main dashboard to immediately start pulling information from a database [15], but a database needed to be created in AWS (Amazon Web Services) before the data could be displayed. Linux server used in DevOps earlier in 2025 that was installed on the current laptop, this served as a “test server” before moving anything onto another EC2 instance for simplicity.

Using old instructions from a DevOps lab, MongoDB was installed on the “database” server [3]. The dashboard connected to the database, where to create or read data from.

Having a test database set up in AWS was a good milestone, and then tested whether metrics could be pulled from another EC2 instance and populate that data into the database server. The next Linux EC2 instance was created and named “metrics”. Not having learned MongoDB python syntax, this was researched and tested by creating a test script on the test server, it was found that it was possible to populate the database via the script [16], This answers the question of how to populate the database with a Python script.

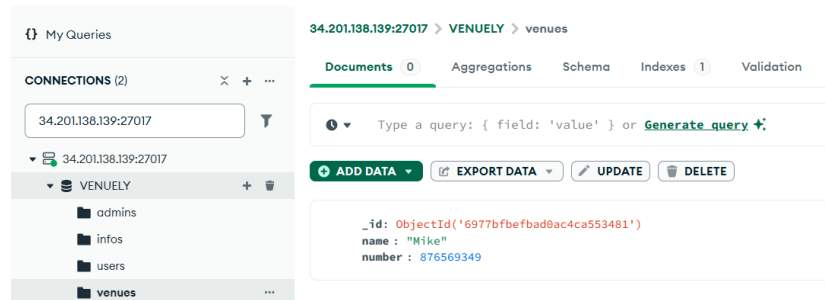


Figure 8 - First attempt at storing data in MongoDB via Python

Going back to the graph on the dashboard, this was then edited to pull data from the database, The below image (figure 9) shows how data can be moved using a python script and the current repository:

```
new Chart(ctx, {
  type: 'bar',
  data: {
    labels: [{{#each venues}}{{name}}{{#unless @last}}, {{/unless}}{{/each}}],
    datasets: [{
      label: '# of Votes',
      data: [{{#each venues}}{{number}}{{#unless @last}}, {{/unless}}{{/each}}],
      borderWidth: 1
    }]
  },
  options: {
```

Figure 9 - Use of handlebars syntax to present data in chart

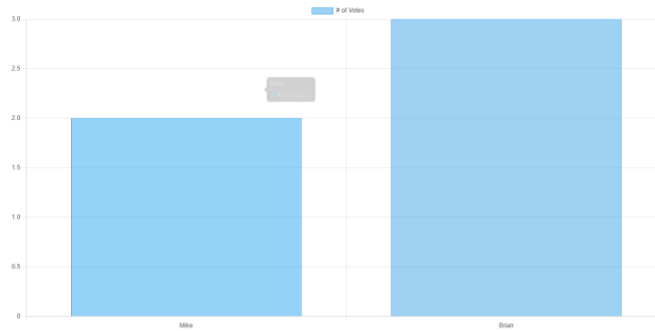


Figure 10 – Displayed result from figure 8

ChatGPT was then used to generate a command that returned a single value for the CPU of the machine on which it was run. Next step was to change the chart to a line graph, then add the following to update the line chart for live updates, which is crucial for real-time updates [19]:

```
setInterval(() => {
  location.reload();
}, 30000);
```

Figure 11 - Reload page every 30 seconds

2 bits of information were added based on uptime and memory usage [22], referenced from an old DevOps assignment, used Bulma to create 2 separate boxes for the uptime and memory used [20]. Manipulated strings in the Dashboard controller, to give the latest uptime and memory used to display on the dashboard [21].

```
test.py
1 import pymongo
2 import subprocess #used to run commands from python
3 import time #used for the delay to wait for the web server to be up and running
4 from datetime import datetime
5 cpu_cmd = [
6     "ssh",
7     "-t", "Mike.pem",
8     "ec2-user@34.228.112.10",
9     "top -bni | grep 'Cpu(s)' | awk '{print 100 - $8}'"
10 ]
11
12 uptime_cmd = [
13     "ssh",
14     "-t", "Mike.pem",
15     "ec2-user@34.228.112.10",
16     "uptime -p"
17 ]
18
19 mem_cmd = [
20     "ssh",
21     "-t", "Mike.pem",
22     "ec2-user@34.228.112.10",
23     "free -m | awk 'NR==2{printf \"%2f%%\", $3*100/$2}'"
24 ]
25
26 myclient = pymongo.MongoClient("mongodb://54.86.133.49:27017/")
27 #print(myclient.list_database_names())
28 mydb = myclient["ODIN"]
29 mycol = mydb["InstanceStats"]
30
31 while(True):
32     cpu = subprocess.run(cpu_cmd, capture_output=True, text=True)
33     uptime = subprocess.run(uptime_cmd, capture_output=True, text=True)
34     memUsed = subprocess.run(mem_cmd, capture_output=True, text=True)
35     mydict = {"time": datetime.now(), "CPU": cpu.stdout.strip(), "uptime": uptime.stdout.strip(), "memUsed": memUsed.stdout.strip()}
36     x = mycol.insert_one(mydict)
37     time.sleep(30)
```

Figure 12 - Initial python collector script

The script above is built to just “collect data”.

Automated Action on collected Metric Data

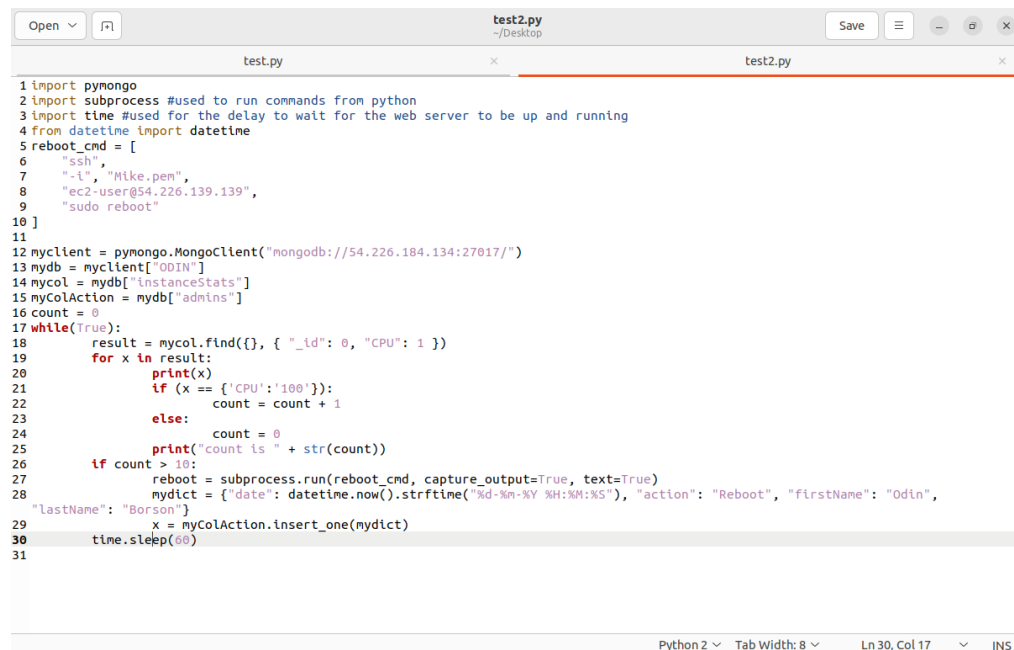
Now that metrics on the EC2 instances are being populated into a database, it was found that a separate python script could act on that data, it will automatically try to reboot if there are more than 10 instances of CPU usage above 100:



```
1 import pymongo
2 import subprocess #used to run commands from python
3 import time #used for the delay to wait for the web server to be up and running
4 from datetime import datetime
5 reboot_cmd = [
6     "ssh",
7     "-i", "Mike.pem",
8     "ec2-user@34.228.112.10",
9     "sudo reboot"
10 ]
11
12 myclient = pymongo.MongoClient("mongodb://54.86.133.49:27017/")
13 mydb = myclient["ODIN"]
14 mycol = mydb["instanceStats"]
15 count = 0
16 while(True):
17     result = mycol.find({}, { "_id": 0, "CPU": 1 })
18     for x in result:
19         print(x)
20         if (x == {'CPU': '100'}):
21             count = count + 1
22     if count > 10:
23         reboot = subprocess.run(reboot_cmd, capture_output=True, text=True)
24         print(reboot.stderr)
25         time.sleep(3000)
26
```

Figure 13 - Rule Based approach to performing actions on data

It's important to keep a log of any actions performed on the system, so the python script was adjusted to populate the Event log on the dashboard whenever a reboot is executed, fixed the date format also [23]:



```
1 import pymongo
2 import subprocess #used to run commands from python
3 import time #used for the delay to wait for the web server to be up and running
4 from datetime import datetime
5 reboot_cmd = [
6     "ssh",
7     "-i", "Mike.pem",
8     "ec2-user@54.226.139.139",
9     "sudo reboot"
10 ]
11
12 myclient = pymongo.MongoClient("mongodb://54.226.184.134:27017/")
13 mydb = myclient["ODIN"]
14 mycol = mydb["instanceStats"]
15 mycolAction = mydb["admins"]
16 count = 0
17 while(True):
18     result = mycol.find({}, { "_id": 0, "CPU": 1 })
19     for x in result:
20         print(x)
21         if (x == {'CPU': '100'}):
22             count = count + 1
23         else:
24             count = 0
25     print("count is " + str(count))
26     if count > 10:
27         reboot = subprocess.run(reboot_cmd, capture_output=True, text=True)
28         mydict = {"date": datetime.now().strftime("%d-%m-%Y %H:%M:%S"), "action": "Reboot", "firstName": "Odin",
29                 "lastName": "Borson"}
30         x = mycolAction.insert_one(mydict)
31         time.sleep(60)
32
```

Figure 14 - Further additions to figure 12

Below is the dashboard's look and feel so far, a button was added that will go red and say "bad" when CPU is high for 5 instances of 100% CPU, and to update the dashboard to advise when a reboot has been automatically been performed. This served as a good milestone covered, it's possible to collect and act on data:

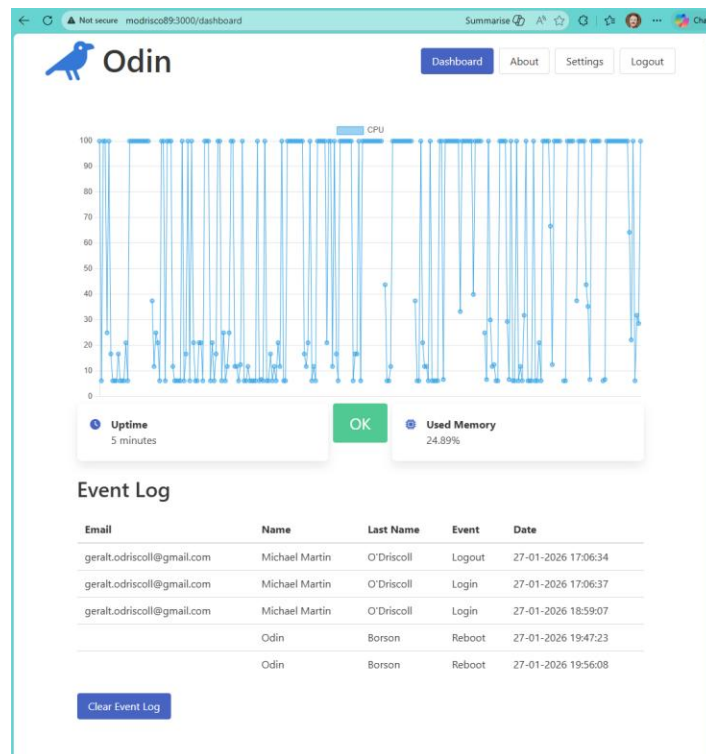


Figure 15 - Dashboard

AI-Driven comments and System Administrator Control

Since good progress has been made, an investigation was conducted on how to create a Python AI agent and to see how it could be used [24]. Generated an Open AI API key (this costs €10 minimum). The same code from the video was copied from GitHub resource mentioned in the YouTube video to confirm functionality [24] [28]. The code from the GitHub resource worked, then an idea came to mind, to adjust the code so as, not to have a conversation with the AI Agent, but to have the AI agent prompt itself every 2 minutes and populate its responses in a database and then populate the information on the dashboard.

In other words, the AI is looking at the data and commenting on the system's performance.

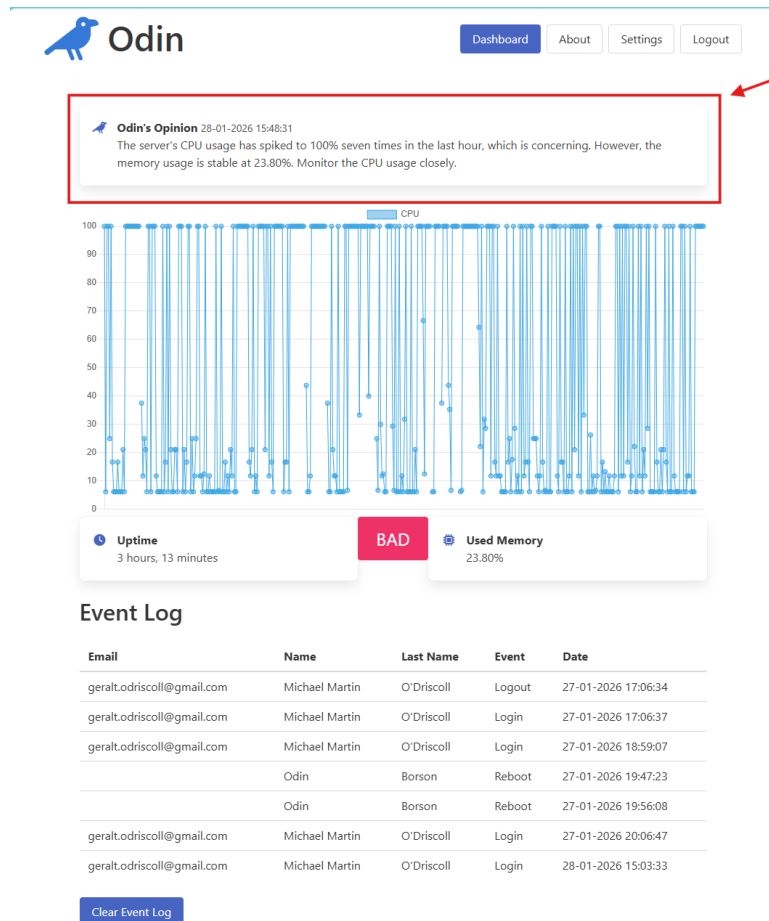


Figure 16 - Automated opinion from Odin

The Agent was adjusted through trial and error, since there's a function which contains the reboot script, the rule based approach seemed to be inferior to the Agent at this point [29][16] It was found that the @tool functions are basically tools for the AI to utilize, in performing an "action" which what was set out for in the first place.

An additional view was added called "Opinion Log", this loads up a table of all opinions added to the database from the AI Agent:

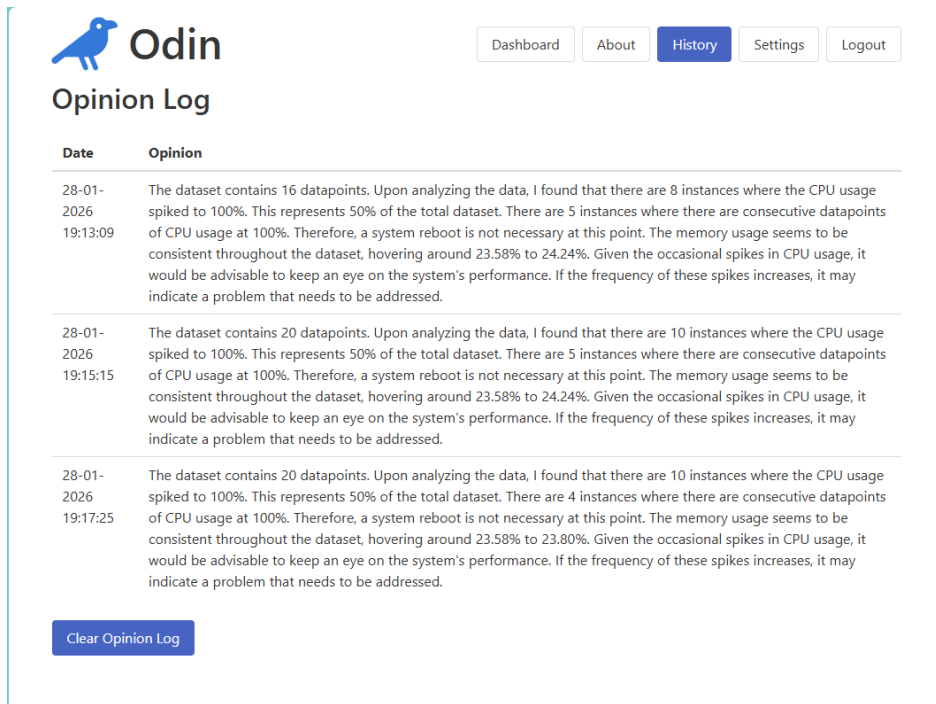


Figure 17 - Opinion Log

Added an additional Doughnut chart for “memory usage” and a pie chart for “database storage usage” [33][34], this meant the collector script needed to be updated to include accessing the database and populating it with the value of the storage.

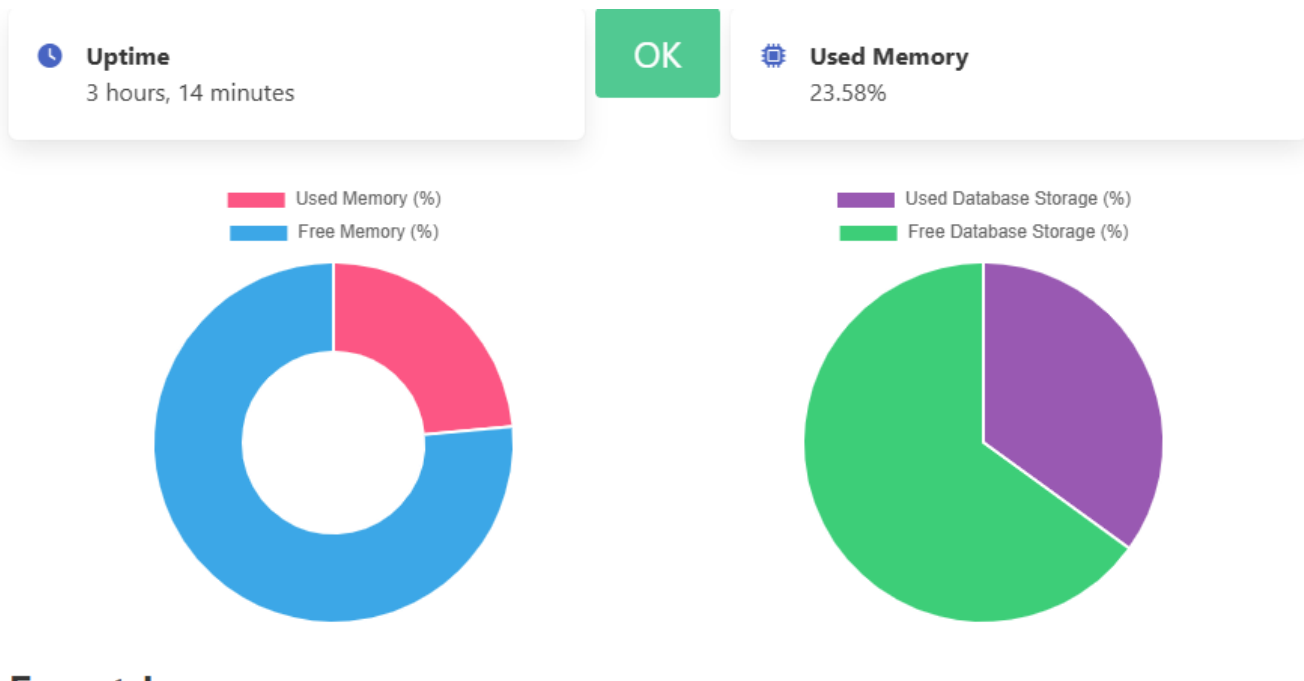


Figure 18 – Memory Usage and Database Storage Usage presented

This meant another command was inserted into the collector script to collect the latest log, and then tried to split the comments from Odin into 3 separate parameters, however this drove up the spending usage on OpenAI, so kept it to one prompt, it's possible there's too much info in the lastLog parameter

where the last log was stored. Decided to boycott that idea of looking at logs, since it's costly in OpenAI, it increased the size of the dataset, which increased cost.

Running Python Scripts from JavaScript

ChatGPT was used to understand how to run the dashboard from the test environment as opposed to the laptop, the reasoning behind this was to enable System Administrator to perform actions from the dashboard, easiest way to do this was to run the dashboard from the test environment where the scripts were ran.

Odin repository was cloned to the test environment for testing and it was successfully ran from there. The next step was to work out how to run python scripts from Java.

A Button was created on the dashboard that will only appear when conditions are met, this button runs a reboot script in python in the same dashboard directory.

Monitoring multiple Instances

This was the challenging section whereby multiple EC2 instances were being monitored.

Firstly the Full-Stack VPC containing the 3 Server Instances and 1 database instances were created, with an Elastic Load balancer and autoscaling (1 EC2 server gets terminated, autoscaling would automatically replace the terminated instance).

Upon completion of the Full-Stack VPC, a new collector script was created called "CollectorV2", the reason another script was created to enable the option to switch between "Collector" and "CollectorV2", the first being the single instance monitoring script, the latter for multiple.

Unfortunately it was difficult to find resources to assist with development of CollectorV2, so ChatGPT was prompted for assistance, refer to Appendices for these prompts.

Conclusion

As the title suggests, “An Autonomous Cloud Infrastructure Management System for Monitoring, Analysing, and Responding to Operational Events Ensuring Reliability, Scalability and Resilience Across Multiple Cloud Instances”. Most project milestones were achieved where the following was implemented:

- Monitoring EC2 instances via collection of parameter data (CPU, database storage, used memory, etc.)
- Adjusting an AI agent for this project so that it would analyse parameter data and discuss it, and even act on it, which was a stretch goal. Originally it would act on the data automatically, but in a real-world scenario, a human would need to be kept in the loop to make the decision on what action to take.
- Created a “Full-Stack VPC” which was monitored, meaning multiple instances were monitored. Although extra features were found to be too difficult to achieve in the time given, features like the ability to reboot an unhealthy instance from the dashboard.

As mentioned, the full functionality that was intended was not completed, but the idea of an AI agent commenting on the data was not planned for, but felt appropriate and considered a nice feature to have, so the project ended up a little different.

The plan was to follow the Gantt chart in the planning section of this document. This had a bottom-up approach, but instead a top-down approach was executed, mainly due to the bigger interest in collecting data, acting on the data, and even using an AI agent to analyse the data, which was surprisingly not as difficult as once thought.

A major lesson to all this is “good time management”. With intense work responsibilities, and other responsibilities outside of SETU, if another chance was given to execute this project, Google Calendar would have been used to map out the work as opposed to a Gantt chart. The reason being, Google Calendar is a lot easier to use.

More attention could have been paid to the market analysis, and repository analysis. Not much inspiration was taken from this research.

On a positive note, a lot of knowledge has been gained, and a fear of the unknowns such as “AI agent” has diminished and opens doors to a lot of excitement of what else is out there to learn.

References

- [1] Modrisco89 (n.d.) *Full-Stack-Assignment-1*. GitHub repository. Available at: <https://github.com/modrisco89/Full-Stack-Assignment-1> (Accessed: 27 January 2026).
- [2] Tutors.dev (2025) *Topic 07 – AWS VPC*. Available at: <https://tutors.dev/topic/setu-hdip-comp-sci-2024-devops/topic-07-AWS-VPC> (Accessed: 10 January 2026).
- [3] Tutors.dev (2025) *Lab – Deploying a Node app on AWS*. Available at: <https://tutors.dev/lab/setu-hdip-comp-sci-2024-devops/topic-10-Security/unit-2-node-deployment/book-1-Deploying-a-node-app> (Accessed: 10 January 2026).
- [4] Tutors.dev (no date) *boto3*. Available at: <https://tutors.dev/lab/setu-hdip-comp-sci-2024-devops/topic-04-Interacting-with-AWS-boto3/unit-2-boto3/book-1-boto3> (Accessed: 11 January 2026).
- [5] YouTube (2022)* *AWS EC2 instances | Replication in MongoDB*. Available at: <https://www.youtube.com/watch?v=nwpWaqmqNSE> (Accessed: 12 January 2026).
- [6] Tutors.dev (2025) *WIT HDip Comp Sci 2024 Full Stack 1*. Available at: <https://www.tutors.dev/course/wit-hdip-comp-sci-2024-full-stack-1> (Accessed: 11 January 2026).
- [7] Nagios.org (no date) *Nagios – Open Source Network Monitoring Solution*. Available at: <https://www.nagios.org/> (Accessed: 11 January 2026).
- [8] Nagios.org (no date) *Nagios Core – Open source IT infrastructure monitoring tool*. Available at: <https://www.nagios.org/projects/nagios-core/> (Accessed: 11 January 2026).
- [9] Zabbix.com (no date) *Zabbix Overview & Demo*. Available at: <https://www.zabbix.com/demo> (Accessed: 12 January 2026).
- [10] agenticsorg (no date) *devops: A fully autonomous, AI-powered DevOps platform for managing cloud infrastructure*. GitHub repository. Available at: <https://github.com/agenticsorg/devops> (Accessed: 13 January 2026).
- [11] Google Cloud (2025) *What are AI agents? Definition, examples, and types*. Available at: <https://cloud.google.com/discover/what-are-ai-agents> (Accessed: 14 January 2026).
- [12] fzaninotto (no date) *uptime: A remote monitoring application using Node.js, MongoDB, and Twitter Bootstrap*. GitHub repository. Available at: <https://github.com/fzaninotto/uptime> (Accessed: 29 January 2026).
- [13] KSJaay (no date) *Lunalytics: Open source monitoring tool built with Node.js & React*. GitHub repository. Available at: <https://github.com/KSJaay/Lunalytics> (Accessed: 14 January 2026).
- [14] Uwins, L. (no date) *Blue: The branding colour that builds trust and lasting impressions*. Soley Creative. Available at: <https://www.soleycreative.com/studio-notes/blue-colour-psychology-culture> (Accessed: 27 January 2026).

- [15] Chart.js (2025) *Getting Started with Chart.js*. Available at: <https://www.chartjs.org/docs/latest/getting-started/> (Accessed: 27 January 2026)
- [16] W3Schools (no date) *Python MongoDB Get Started*. Available at: https://www.w3schools.com/python/python_mongodb_getstarted.asp (Accessed: 27 January 2026).
- [17] Modrisco89 (no date) *Odin*. GitHub repository. Available at: <https://github.com/modrisco89/Odin> (Accessed: 27 January 2026).
- [18] Python Software Foundation (2026) *datetime — Basic date and time types*. Available at: <https://docs.python.org/3/library/datetime.html> (Accessed: 27 January 2026).
- [19] RaceBase (2012) *Refresh Page for interval using js*. Stack Overflow, 20 August 2012. Available at: <https://stackoverflow.com/questions/12038183/refresh-page-for-interval-using-js> (Accessed: 27 January 2026).
- [20] Bulma (no date) *Box*. Bulma: Free, open source, and modern CSS framework based on Flexbox. Available at: <https://bulma.io/documentation/elements/box/> (Accessed: 27 January 2026).
- [21] JavaScript Cheatsheet (no date) *Javascript String Manipulation*. Available at: <https://www.javascriptcheatsheet.org/cheatsheet/manipulating-strings> (Accessed: 27 January 2026).
- [22] Michael O'Driscoll (2025) *Assignment 1 DevOps*. [Unpublished assignment, South East Technological University].
- [23] w3resource (2025) *Python: Print current time and date*. Available at: <https://www.w3resource.com/python-exercises/python-basic-exercise-3.php> (Accessed: 27 January 2026).
- [24] Build a Python AI Agent in 10 Minutes (2025) *YouTube*, 26 October. Available at: <https://www.youtube.com/watch?v=gaGaPpnexxA> (Accessed: 28 January 2026).
- [25] LangChain (no date) *Install LangChain: LangChain OSS Python documentation*. Available at: <https://docs.langchain.com/oss/python/langchain/install> (Accessed: 29 January 2026).
- [26] LangChain (no date) *LangGraph overview: LangChain OSS Python documentation*. Available at: <https://docs.langchain.com/oss/python/langgraph/overview> (Accessed: 29 January 2026).
- [27] OpenAI (no date) *API keys: OpenAI Platform*. Available at: <https://platform.openai.com/api-keys> (Accessed: 29 January 2026).
- [28] techwithtim (2025) *main.py in PythonAIAgentin10Minutes repository*. GitHub. Available at: <https://github.com/techwithtim/PythonAIAgentin10Minutes/blob/main/main.py#L6> (Accessed: 29 January 2026).
- [29] Irfan (2013) *TypeError: ObjectId("") is not JSON serializable*, Stack Overflow. Available at: <https://stackoverflow.com/questions/16586180/typeerror-objectid-is-not-json-serializable> (Accessed: 29 January 2026)

- [30] Handlebars.js (2026) “@data variables”. *Handlebars.js API Reference*. Available at: <https://handlebarsjs.com/api-reference/data-variables.html#last> (Accessed: 27 January 2026).
- [31] Python Software Foundation (2026) “*subprocess.run — Run a command, wait for completion, then return a CompletedProcess instance*”. *Python 3 Library Reference*. Available at: <https://docs.python.org/3/library/subprocess.html#subprocess.run> (Accessed: 27 January 2026).
- [32] Great Learning Editorial Team (2025) “*Strip() Function in Python with Examples*”. *MyGreatLearning*. Updated 6 January 2025. Available at: <https://www.mygreatlearning.com/blog/strip-in-python/> (Accessed: 27 January 2026).
- [33] Chart.js (2024) *Responsive configuration*. Available at: <https://www.chartjs.org/docs/latest/configuration/responsive.html> (Accessed: 10 February 2026).
- [34] Chart.js (2025) *Doughnut and Pie Charts*. Available at: <https://www.chartjs.org/docs/latest/charts/doughnut.html> (Accessed: 10 February 2026).
- [35] Stack Overflow (2009) *How can I convert a string to an integer in JavaScript?* Available at: <https://stackoverflow.com/questions/1133770/how-can-i-convert-a-string-to-an-integer-in-javascript> (Accessed: 10 February 2026).
- [36] JavaScriptCheatSheet.org (n.d.) *Javascript String Manipulation*. Available at: <https://www.javascriptcheatsheet.org/cheatsheet/manipulating-strings> (Accessed: 10 February 2026).
- [37] Bulma (n.d.) *Level – Layout*. Available at: <https://bulma.io/documentation/layout/level/> (Accessed: 10 February 2026).
- [38] Amazon Web Services (n.d.) *Amazon EC2 key pairs and Amazon EC2 instances*. Available at: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-key-pairs.html> (Accessed: 10 February 2026).
- [39] Running a Python File by Running a JavaScript File (2024) *YouTube video* 6 January. Available at: <https://www.youtube.com/watch?v=aEeq1dfB8Zg> (Accessed: 15 February 2026).
- [40] Gadam, S., 2023. *What is iterative and incremental development? Process, examples*. LogRocket Blog, 26 January. Available at: <https://blog.logrocket.com/product-management/what-is-iterative-incremental-development-process-examples/>

Appendices

LLM Prompts

how can I obtain current cpu usage as a single number in a linux instance

You want a **single number representing current CPU usage** on a Linux system. There are several ways, depending on how precise or simple you want it. I'll break it down step by step.

1. Using top in batch mode

top can output CPU usage in a one-shot, scriptable way:

```
top -bn1 | grep "Cpu(s)" | awk '{print 100 - $8}'
```

Explanation:

- top -bn1: run top in **batch mode** for 1 iteration.
 - grep "Cpu(s)": grab the CPU summary line.
 - \$8 is %id (idle percentage), so 100 - \$8 gives **CPU usage percentage**.
-

2. Using mpstat

If sysstat is installed:

```
mpstat 1 1 | awk '/Average/ {print 100 - $12}'
```

Explanation:

- \$12 is %idle, so subtracting it from 100 gives CPU usage.
 - 1 1 means sample for 1 second once.
-

3. Using /proc/stat (direct method)

You can compute CPU usage from /proc/stat:

```
#!/bin/bash
```

```
# Read initial CPU stats
```

```
read cpu user nice system idle iowait irq softirq steal guest < /proc/stat
```

```
total1=$((user+nice+system+idle+iowait+irq+softirq+steal))
```

```
idle1=$((idle+iowait))
```

```
# Wait a short interval
```

```
sleep 1
```

```
# Read CPU stats again
```

```
read cpu user nice system idle iowait irq softirq steal guest < /proc/stat
```

```
total2=$((user+nice+system+idle+iowait+irq+softirq+steal))
```

```
idle2=$((idle+iowait))
```

```
# Calculate usage
```

```
cpu_usage=$((100 * ((total2 - total1) - (idle2 - idle1)) / (total2 - total1))
```

```
echo $cpu_usage
```

This outputs a **single integer percentage** representing current CPU usage.

✅ **Quickest way (if you have top):**

```
top -bn1 | grep "Cpu(s)" | awk '{print 100 - $8}'
```

If you want, I can also give a **one-liner using only /proc/stat** without a full script—very lightweight and perfect for monitoring scripts.

Do you want me to do that?

using df -h how to get use% as a value outputted with a command

You can grab the **Use%** column from df -h with a little text processing. Here are the clean, common ways—pick your vibe 😊

Get Use% for a specific mount (most common)

Example: root filesystem /

```
df -h / | awk 'NR==2 {print $5}'
```

Output:

42%

Why it works:

- NR==2 skips the header
 - \$5 is the **Use%** column
-

Get Use% without the percent sign

Useful for scripts / comparisons:

```
df -h / | awk 'NR==2 {gsub("%","", $5); print $5}'
```

Output:

42

Get Use% for all mounted filesystems

```
df -h | awk 'NR>1 {print $6, $5}'
```

Output:

/ 42%

/boot 18%

/home 77%

(\$6 = mount point, \$5 = Use%)

Script-friendly / safer version (POSIX, no -h dependency)

If you don't care about human-readable sizes:

```
df / | awk 'NR==2 {print $5}'
```

Bonus: use df --output (cleanest)

GNU df supports this and it's *chef's kiss* for scripting:

```
df --output=pcent / | tail -1
```

Output:

42%

Strip whitespace if needed:

```
df --output=pcent / | tail -1 | tr -dc '0-9%'
```

If you tell me:

- **which filesystem** (/ , /home, Docker volume, etc.)
- **whether this is for a script, monitoring, or one-off check**

...I'll tailor the exact command for you.

cmd to check os on linux

Here are the quickest commands to check your OS on Linux:

✅ 1. Most Common (Works on almost all distros)

```
cat /etc/os-release
```

Example output:

```
NAME="Amazon Linux"
```

```
VERSION="2023"
```

```
ID="amzn"
```

✅ 2. Short & Clean Output

```
lsb_release -a
```

If not installed:

```
sudo yum install redhat-lsb -y # Amazon / RHEL
```

```
sudo apt install lsb-release -y # Ubuntu / Debian
```

✅ 3. Check Kernel Version

```
uname -a
```

or shorter:

```
uname -r
```

✅ 4. See if You're on Amazon Linux (common for EC2)

```
cat /etc/system-release
```

🔥 Fastest one-liner I personally use:

```
cat /etc/os-release | head -n 3
```

If you're on EC2 and want, paste your output here and I'll tell you exactly what environment you're running.

can I do the same in the first message on ubuntu 22.04.5

Yes — **everything from the first answer works perfectly on Ubuntu 22.04.5** 👍

Ubuntu fully supports the standard Linux Git workflow.

Here's exactly how it applies to **Ubuntu 22.04.5** on EC2:

✅ Option 1 (git pull on server) — Works 100% on Ubuntu

1 Install git (if needed)

```
sudo apt update
```

```
sudo apt install git -y
```

2 Clone your repo

```
git clone https://github.com/yourusername/yourrepo.git
```

```
cd yourrepo
```

3 Every time you update locally

On your laptop:

```
git push origin main
```

On your Ubuntu EC2:

```
cd yourrepo
```

```
git pull origin main
```

Done.

🚀 Even Faster (Run from your laptop)

```
ssh ubuntu@your-ec2-ip "cd yourrepo && git pull origin main"
```

That works perfectly on Ubuntu AMIs.

🔒 If Your Repo Is Private (Recommended Setup)

Use SSH deploy keys:

On EC2:

```
ssh-keygen -t ed25519
```

```
cat ~/.ssh/id_ed25519.pub
```

Add that key to your GitHub repo:

Settings → Deploy Keys

Then clone using:

```
git clone git@github.com:yourusername/yourrepo.git
```

Now pulls won't ask for passwords.

⚡ If You Want Auto Deploy on Push

Ubuntu works great with:

- GitHub Actions
 - or a simple webhook + deploy script
 - or systemd/PM2 restart after pull
-

If you tell me what you're running (Node, Python, Django, etc.), I can give you a clean Ubuntu-specific deploy script you can copy-paste.

```
sudo yum install -y nodejs
```

That command:

```
sudo yum install -y nodejs
```

✗ Will NOT work on Ubuntu 22.04.5

yum is for:

- Amazon Linux
- RHEL
- CentOS

Ubuntu uses **apt**, not yum.

✅ Correct Way on Ubuntu 22.04.5

⚠ Do NOT use the default Ubuntu repo version

It's usually outdated.

🔥 Best Way (Recommended): Install Node via NodeSource

1 Install Node 20 (LTS – recommended)

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -  
sudo apt install -y nodejs
```

Verify

```
node -v  
npm -v
```

Alternative (Best for managing versions): Use NVM

If this is development or you want flexibility:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash  
source ~/.bashrc  
nvm install 20  
Then:  
node -v
```

Quick Comparison

Method	Best For
apt default repo	Simple but older version
NodeSource	Production EC2 servers
NVM	Dev / multiple Node versions

If this is your EC2 production server, tell me:

- Is it a Node app?
- Are you using PM2?
- Is it behind Nginx?

I'll give you a proper production-ready setup.

dude doesn't understand curl

😄 Got it — let's do this **without curl wizardry**.

You're on **Ubuntu 22.04**, so here are the simple options:

✅ **Easiest (No curl, no scripts)**

Just use Ubuntu's package manager:

```
sudo apt update
```

```
sudo apt install -y nodejs npm
```

Then check:

```
node -v
```

```
npm -v
```

That's it.

👉 Downside: Ubuntu's repo version may be a little older.

✅ **Slightly Better (Still Simple, No curl piping)**

If you want a newer version but don't want to pipe curl into bash:

```
sudo apt update
```

```
sudo apt install -y ca-certificates gnupg
```

Then manually add NodeSource repo:

```
sudo mkdir -p /etc/apt/keyrings
```

```
wget -qO- https://deb.nodesource.com/gpgkey/nodesource-repo.gpg.key | sudo tee  
/etc/apt/keyrings/nodesource.gpg > /dev/null
```

Add the repo:

```
echo "deb [signed-by=/etc/apt/keyrings/nodesource.gpg] https://deb.nodesource.com/node_20.x  
nodistro main" | sudo tee /etc/apt/sources.list.d/nodesource.list
```

Then:

```
sudo apt update
```

```
sudo apt install -y nodejs
```

Ultra-Simple Answer If You Just Want It Working

Run this and move on with life:

```
sudo apt install -y nodejs npm
```

If you tell me what error you're hitting, I'll fix it in one shot.

```
sudo apt install -y nodejs npm reference
```

If you're asking what this command means:

```
sudo apt install -y nodejs npm
```

Here's the breakdown 🙋

Command reference

- **sudo** → Run the command as administrator (root privileges)
 - **apt** → Ubuntu's package manager
 - **install** → Tells apt to install packages
 - **-y** → Automatically answer "yes" to prompts
 - **nodejs** → Installs the Node.js runtime
 - **npm** → Installs Node Package Manager
-

What gets installed?

On **Ubuntu 22.04**, this installs:

- Node.js (from Ubuntu's official repositories)
 - npm
 - Required dependencies
-

Verify installation

After running it:

```
node -v
```

```
npm -v
```

Official references

- Ubuntu apt manual:
- `man apt`
- Package info:
- `apt show nodejs`
- `apt show npm`

If you meant something different by “reference” (like documentation link or version info), tell me what you're looking for and I'll give you exactly that.

where did you get this command

I didn't pull it from a random site — it's the **standard Ubuntu package manager syntax**.

On **Ubuntu 22.04**, software is installed using apt, and nodejs + npm are official packages in Ubuntu's repositories.

You can verify this directly on your machine:

`apt search nodejs`

`apt search npm`

Or check package details:

`apt show nodejs`

`apt show npm`

You'll see they come from Ubuntu's official repos (maintained by Canonical).

So the command:

`sudo apt install -y nodejs npm`

comes from:

- Ubuntu's apt package manager usage
- The fact that nodejs and npm are valid package names in Ubuntu 22.04

It's not a custom trick — it's just normal Ubuntu package installation syntax.

If you're asking because you're worried about version quality, that's a different question — Ubuntu's repo version can be older than the latest Node release.

funny, my project is on ubuntu but doesn't work on it!

😂 Okay — this is almost certainly a **Node version mismatch** problem.

Ubuntu 22.04 installs an older Node version by default.

Let's confirm first.

Step 1 — Check your Node version on Ubuntu

Run:

```
node -v
```

Now check your laptop:

```
node -v
```

If they're different (especially if Ubuntu is older), that's your problem.

Why This Happens

Ubuntu 22.04 repo usually installs something like:

- Node 12 / 14 / 16 (depending on updates)

But most modern projects require:

- Node 18+
- Node 20 (current LTS)

If your project was built using Node 20 locally and Ubuntu has Node 14 — things break.

🔥 Clean Fix (No curl piping, simple method)

Remove old node:

```
sudo apt remove nodejs npm -y
```

Install **nvm** (Node Version Manager):

```
sudo apt install build-essential -y
```

Then:

```
wget -qO- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
```

```
source ~/.bashrc
```

Install the same version you use locally:

```
nvm install 20
```

```
nvm use 20
```

Now check:

```
node -v
```

💡 Even Better

On your laptop:

```
node -v
```

Then on Ubuntu:

```
nvm install <that exact version>
```

Boom — identical environment.

If it still doesn't work, paste:

- `node -v`
- the error message
- what framework you're using (Express? Next? React build? etc.)

We'll fix it properly instead of guessing.

first is 12.22.9, laptop is v24.11.0

😄 There it is.

Your server is running:

12.22.9 ← ancient

Your laptop is running:

v24.11.0 ← bleeding edge

That's not a "small mismatch."

That's **2021 vs 2026 technology**.

Node 12 is **end-of-life** and will absolutely break modern projects.

🔥 The Real Fix (Do This)

You need to install Node 24 on Ubuntu.

Step 1 — Remove old Node

```
sudo apt remove nodejs npm -y
```

Step 2 — Install Node 24 (Proper Way)

```
curl -fsSL https://deb.nodesource.com/setup_24.x | sudo -E bash -  
sudo apt install -y nodejs
```

Step 3 — Confirm

```
node -v
```

It should now say:

v24.11.0 (or close)

⚠️ Important Production Advice

Node 24 is very new.

For production servers, most people use:

- Node 20 (LTS, stable)

Unless you specifically need Node 24 features, consider matching your dev environment to LTS instead.

Why Your App Broke

Node 12:

- Doesn't support modern ES modules properly
- Doesn't support many newer JS features
- Fails on modern dependency builds
- Breaks most current frameworks

So your project wasn't "broken on Ubuntu."

It was just running on a fossil 🦕

If you want, tell me:

- Is this production?
- What framework?
- Do you use PM2 or just node server.js?

I'll give you a clean deploy setup.

how do I install curl??

Ah, got it — your Ubuntu 22.04 server probably **doesn't have curl installed** by default. That's easy to fix.

✅ Install curl on Ubuntu 22.04

```
sudo apt update
```

```
sudo apt install -y curl
```

✅ Verify

```
curl --version
```

You should see something like:

```
curl 8.2.1 (x86_64-pc-linux-gnu) ...
```

Once curl is installed, you can use it to run the NodeSource installer safely:

```
curl -fsSL https://deb.nodesource.com/setup_24.x | sudo -E bash -
```

```
sudo apt install -y nodejs
```

💡 Tip: curl is tiny and safe — it's just a tool to download files or scripts over HTTP/HTTPS.

If you want, I can give you a **single copy-paste command sequence** that will install curl, Node 24, npm, and confirm the version — everything ready for your project.

Do you want me to do that?

`curl -fsSL https://deb.nodesource.com/setup_24.x | sudo -E bash - sudo apt install -y nodejs` all one line?

Yep — you **can** run it all in one line, but you need to separate the commands with `&&` so the second runs **only if the first succeeds**.

Here's the safe one-liner for Ubuntu:

`curl -fsSL https://deb.nodesource.com/setup_24.x | sudo -E bash - && sudo apt install -y nodejs`

✅ What this does

1. `curl -fsSL https://deb.nodesource.com/setup_24.x | sudo -E bash -`
 - Downloads and runs the NodeSource setup script
 - Adds Node 24 repo to your system
 2. `sudo apt install -y nodejs`
 - Installs Node.js **from the NodeSource repo**
 - Includes npm automatically
-

🔍 Verify

After running:

`node -v`

`npm -v`

You should now see **v24.11.0** (matching your laptop).

If you want, I can give you a **full EC2-ready command** that also installs curl first, so it's literally copy-paste and done.

Do you want me to do that?

nash command not found

Ah — “nash: command not found” usually happens when you try to run a script that expects **bash**, but your shell is **sh** or something else.

On Ubuntu, the default shell for some scripts can be **dash**, not **bash**. NodeSource scripts require **bash**.

✅ Correct way to run NodeSource install safely

Make sure you explicitly use bash:

```
sudo apt update
```

```
sudo apt install -y curl bash
```

```
curl -fsSL https://deb.nodesource.com/setup_24.x | sudo -E bash -
```

```
sudo apt install -y nodejs
```

◆ Explanation

- bash is the shell that NodeSource expects.
 - Ubuntu's /bin/sh points to dash, which can't run some bash-specific scripts.
 - Installing bash first ensures the setup script works.
-

🔍 Verify

```
node -v
```

```
npm -v
```

Should show **v24.x**.

If you want, I can give you a **single copy-paste command** that:

1. Installs curl & bash if missing
2. Runs Node 24 setup
3. Installs Node.js & npm
4. Verifies version

...so you don't have to worry about any "command not found" errors.

Do you want me to do that?

how do I remove node altogether first, then install

Perfect — let's do a **clean Node reinstall** on Ubuntu 22.04 so your EC2 matches your laptop's Node 24.

1 Remove any old Node / npm

Run:

```
sudo apt remove -y nodejs npm
```

```
sudo apt purge -y nodejs npm
```

```
sudo apt autoremove -y
```

- remove → uninstalls the packages
- purge → removes config files too
- autoremove → cleans any leftover dependencies

At this point, running `node -v` or `npm -v` should return “command not found”.

2 Make sure curl & bash are installed

```
sudo apt update
```

```
sudo apt install -y curl bash
```

3 Install Node 24 properly

```
curl -fsSL https://deb.nodesource.com/setup_24.x | sudo -E bash -
```

```
sudo apt install -y nodejs
```

4 Verify

```
node -v
```

```
npm -v
```

You should see:

```
v24.x.x
```

⚡ Optional: One-liner copy-paste

```
sudo apt remove -y nodejs npm && sudo apt purge -y nodejs npm && sudo apt autoremove -y && sudo  
apt update && sudo apt install -y curl bash && curl -fsSL https://deb.nodesource.com/setup_24.x | sudo -  
E bash - && sudo apt install -y nodejs && node -v && npm -v
```

This does **everything**: removes old Node, installs dependencies, sets up NodeSource, installs Node 24, and prints versions.

If you want, I can also give you a **full EC2 deploy-ready command** that installs Node 24, PM2, and pulls your GitHub repo automatically in one go.

Do you want me to do that?

ENOENT : no such file or directory, open '/home/modrisco89/odin/.env'

Ah — now we're getting somewhere. That error:

ENOENT: no such file or directory, open '/home/modrisco89/odin/.env'

is **not a Node problem** — it means your project expects a **.env file** in your project directory, but it doesn't exist on your Ubuntu server.

Most projects (especially Node apps) use .env to store **environment variables**, like API keys, database URLs, or ports.

◆ How to fix

1 Check your .env locally

On your laptop:

```
cd /path/to/odin
```

```
ls -a
```

You'll probably see .env. Open it:

```
cat .env
```

It looks like:

```
PORT=3000
```

```
DATABASE_URL=mongodb://localhost:27017/mydb
```

```
SECRET_KEY=abc123
```

2 Copy it to your server

On your laptop, run:

```
scp /path/to/odin/.env ubuntu@your-ec2-ip:/home/modrisco89/odin/.env
```

- Replace `ubuntu@your-ec2-ip` with your server user & IP
 - Replace `/home/modrisco89/odin/` if your server project path is different
-

3 Verify on server

```
ls -a /home/modrisco89/odin
```

You should see `.env`. Then restart your app:

```
node index.js
```

```
# or
```

```
npm start
```

⚠ Notes

- **Do NOT commit `.env` to GitHub** — it often contains secrets.
 - If you don't have `.env` locally, check your project's **README** — some projects require you to create `.env` manually.
-

If you want, I can write a **one-shot EC2 deployment sequence** that:

1. Pulls your repo
2. Installs Node 24
3. Copies `.env` (from a local source or template)
4. Starts your app

...so it's literally one command to get your project running.

Do you want me to do that?

how to check bandwidth of linux network wise

To check **network bandwidth usage in Linux**, there are several tools depending on whether you want **real-time monitoring, per-interface stats, or historical usage**. Here are the most common and useful ones.

1. `iftop` – Real-time bandwidth per connection

Shows which hosts are using your network bandwidth.

```
sudo iftop -i eth0
```

Replace `eth0` with your interface (like `ens33`, `wlan0`, etc.).

Install:

- Ubuntu/Debian

```
sudo apt install iftop
```

What you see:

- Source → Destination traffic
- Current bandwidth usage
- 2s / 10s / 40s averages

2. `nload` – Simple bandwidth monitor

Displays incoming and outgoing traffic with graphs.

```
nload
```

Install:

```
sudo apt install nload
```

Example output:

- Incoming traffic
- Outgoing traffic
- Graphical usage bars

Good for **quick interface bandwidth monitoring**.

3. `vnstat` – Historical bandwidth statistics

Tracks usage over **hours, days, and months**.

Install:

```
sudo apt install vnstat
```

Start service:

```
sudo systemctl enable vnstat  
sudo systemctl start vnstat
```

Check stats:

```
vnstat
```

Daily usage:

```
vnstat -d
```

Monthly usage:

```
vnstat -m
```

4. `ip -s link` – Built-in Linux command

Shows transmitted and received bytes.

```
ip -s link
```

Example output:

```
RX: bytes  packets  
TX: bytes  packets
```

Run it repeatedly to estimate bandwidth change.

5. `sar` – Detailed network statistics

Part of **sysstat** package.

Install:

```
sudo apt install sysstat
```

Check network stats:

```
sar -n DEV 1
```

This prints **bandwidth every second**.

6. `bmon` – Advanced bandwidth monitor

Interactive bandwidth monitor.

Install:

```
sudo apt install bmon
```

Run:

```
bmon
```

Shows:

- Interface speed
- Current traffic
- Graphs

 Quick recommendation

Purpose	Tool
Real-time traffic per host	iftop
Simple bandwidth graph	nload
Monthly/daily statistics	vnstat
Built-in quick check	ip -s link
Detailed monitoring	bmon

 **Tip:** First check your interface name:

```
ip a
```

Common names:

- eth0
- ens33
- enp0s3
- wlan0

If you want, I can also show:

- **How to measure maximum network speed between two Linux servers (iperf3)**
- **How to check bandwidth usage per process** 

ip -s link how can I measure the bandwidth as one value

`ip -s link` shows **total bytes transferred since boot**, not the bandwidth directly.

To get **bandwidth (bytes/sec or bits/sec)** as a **single value**, you measure the **difference between two samples over time**.

Method 1 — Manual Calculation (concept)

Run twice with a delay:

```
ip -s link show eth0
sleep 1
ip -s link show eth0
```

Look at:

```
RX: bytes
TX: bytes
```

Bandwidth formula:

$$\text{Bandwidth} = (\text{NewBytes} - \text{OldBytes}) / \text{Time}$$

Example:

Time RX bytes

```
t1  100000000
t2  100500000
```

Difference:

500000 bytes in 1 second

Bandwidth:

500000 B/s $\approx$ 4 Mbps

Method 2 — One-line bandwidth measurement (recommended)

This prints **RX and TX bandwidth per second**.

```
IF=eth0; RX1=$(cat /sys/class/net/$IF/statistics/rx_bytes); TX1=$(cat /sys/class/net/$IF/statistics/tx_bytes); sleep 1; RX2=$(cat /sys/class/net/$IF/statistics/rx_bytes); TX2=$(cat /sys/class/net/$IF/statistics/tx_bytes); echo "RX: $((RX2-RX1)) B/s | TX: $((TX2-TX1)) B/s"
```

Example output:

```
RX: 245678 B/s | TX: 10234 B/s
```

Method 3 — Convert to Mbps (clean output)

```
IF=eth0; RX1=$(cat /sys/class/net/$IF/statistics/rx_bytes); sleep 1; RX2=$(cat /sys/class/net/$IF/statistics/rx_bytes); echo "$(( (RX2-RX1)*8/1000000 )) Mbps"
```

Example:

12 Mbps

Method 4 — Continuous bandwidth monitor

```
watch -n1 "cat /proc/net/dev | grep eth0"
```

or a simple loop:

```
while true; do
  RX1=$(cat /sys/class/net/eth0/statistics/rx_bytes)
  sleep 1
  RX2=$(cat /sys/class/net/eth0/statistics/rx_bytes)
  echo "$((RX2-RX1)) B/s"
done
```

💡 **Best practical tool (instead of manual calculation):**

```
sar -n DEV 1
```

or

```
nload
```

These directly show **bandwidth/sec**.

✅ If you want, I can also show a **very small 5-line bash script that prints total bandwidth (RX+TX) in Mbps every second** — useful for server monitoring.

keeps saying 0 mbps

If your script keeps showing 0 **Mbps**, it's usually because of **integer rounding** in bash. When the traffic is **less than 1 Mbps**, the calculation:

```
((value*8/1000000))
```

gets rounded down to 0.

Fix: Show decimal Mbps using `awk`

```
IF=eth0
RX1=$(cat /sys/class/net/$IF/statistics/rx_bytes)
sleep 1
RX2=$(cat /sys/class/net/$IF/statistics/rx_bytes)

awk "BEGIN {printf \"%.3f Mbps\\n\", ($RX2-$RX1)*8/1000000}"
```


Example output:

0.245 Mbps

Now you'll see **small bandwidth values instead of 0**.

Also check your interface name

Sometimes the interface is not `eth0`.

Run:

```
ip link
```

You might see names like:

- `ens33`
- `enp0s3`
- `eth0`
- `wlan0`

Use that name in the script:

```
IF=enp0s3
```

Verify traffic is actually happening

Try generating traffic:

```
ping google.com
```

or

```
curl google.com > /dev/null
```

Then run the bandwidth script again.

Simple continuous monitor (recommended)

```
IF=eth0
while true; do
  RX1=$(cat /sys/class/net/$IF/statistics/rx_bytes)
  sleep 1
  RX2=$(cat /sys/class/net/$IF/statistics/rx_bytes)
  awk "BEGIN {printf \"RX: %.3f Mbps\n\", ($RX2-$RX1)*8/1000000}"
done
```

 If you'd like, I can also show:

- A 1-line command that prints total bandwidth (RX+TX) every second
- How to measure max bandwidth between two Linux servers with `iperf3`
- How to see bandwidth per process (which program uses network).

+++++

boto3 to access loadbalancer

To access **AWS Load Balancers** using **Boto3**, you typically use the **Elastic Load Balancing clients**:

- `elbv2` → for **Application Load Balancer (ALB)** and **Network Load Balancer (NLB)**
- `elb` → for **Classic Load Balancer**

Most modern setups use **ALB/NLB**, so you'll usually work with `elbv2`.

1. Setup Boto3

Install and configure credentials.

```
pip install boto3
aws configure
```

2. Create the ELBv2 Client

```
import boto3
```

```
client = boto3.client('elbv2')
```

You can also specify region:

```
client = boto3.client('elbv2', region_name='eu-west-1')
```

3. List Load Balancers

```
response = client.describe_load_balancers()
```

```
for lb in response['LoadBalancers']:
```

```
    print(lb['LoadBalancerName'], lb['DNSName'], lb['Type'])
```

Example output:

```
my-app-alb my-app-alb-123.eu-west-1.elb.amazonaws.com application
```

Useful fields:

- LoadBalancerName
 - LoadBalancerArn
 - DNSName
 - Type
 - State
-

4. Get Details for One Load Balancer

```
response = client.describe_load_balancers(  
    Names=['my-app-alb']  
)  
  
print(response['LoadBalancers'][0])
```

5. Get Target Groups of a Load Balancer

```
response = client.describe_target_groups(  
    LoadBalancerArn='your-lb-arn'  
)  
  
for tg in response['TargetGroups']:  
    print(tg['TargetGroupName'], tg['Port'])
```

6. Get Health of Targets

```
response = client.describe_target_health(  
    TargetGroupArn='your-target-group-arn'  
)  
  
for target in response['TargetHealthDescriptions']:  
    print(target['Target']['Id'], target['TargetHealth']['State'])
```

Example output:

```
i-123456 healthy  
i-456789 unhealthy
```

7. Get Listeners (Ports / HTTPS / HTTP)

```
response = client.describe_listeners(  
    LoadBalancerArn='your-lb-arn'  
)  
  
for listener in response['Listeners']:  
    print(listener['Protocol'], listener['Port'])
```

8. Example: Find LB DNS Name

```
import boto3  
  
client = boto3.client("elbv2")  
  
def get_lb_dns(name):  
    response = client.describe_load_balancers(Names=[name])  
    return response["LoadBalancers"][0]["DNSName"]  
  
print(get_lb_dns("my-app-alb"))
```

Tip:

If you want, I can also show how to:

- Filter **load balancers by tag**
- Map **ALB → target groups → EC2 instances**
- Get **listener rules (routing paths)**
- Or build a **complete AWS inventory script for load balancers**.

thing is I'm monitoring ec2 instances by IP, is it possible to monitor by dns name?

Yes, you **can monitor by DNS name**, but there are some important differences compared with monitoring EC2 instances by IP.

When you use a **Load Balancer DNS name**, you're monitoring the **service endpoint**, not the individual instances behind it.

For example, an **Application Load Balancer** like my-app-alb-123.eu-west-1.elb.amazonaws.com distributes traffic to multiple EC2 instances.

1. Monitor a Load Balancer by DNS (Simple HTTP/TCP check)

If your monitoring tool can resolve DNS, you can monitor the LB endpoint directly.

Example using Python:

```
import requests

url = "http://my-app-alb-123.eu-west-1.elb.amazonaws.com/health"

try:
    r = requests.get(url, timeout=5)
    if r.status_code == 200:
        print("Service healthy")
    else:
        print("Service unhealthy:", r.status_code)
except Exception as e:
    print("Service unreachable:", e)
```

This checks the **actual public service** instead of each instance.

Good for:

- uptime monitoring
 - external health checks
 - synthetic monitoring
-

2. DNS Resolution Monitoring

You can also monitor if DNS resolves correctly.

```
import socket

dns = "my-app-alb-123.eu-west-1.elb.amazonaws.com"
```

```
ip = socket.gethostbyname(dns)
print("Resolved IP:", ip)
```

Note: Load balancer DNS often resolves to **multiple IPs that change frequently**.

3. Using AWS APIs (Recommended for Instance-Level Monitoring)

Instead of monitoring instance IPs directly, you can query target health from the load balancer using **Boto3**.

```
import boto3

client = boto3.client("elbv2")

response = client.describe_target_health(
    TargetGroupArn="target-group-arn"
)

for target in response["TargetHealthDescriptions"]:
    print(
        target["Target"]["Id"],
        target["TargetHealth"]["State"]
    )
```

This tells you if each instance is:

- healthy
- unhealthy
- draining
- unused

This is **more accurate than pinging instance IPs**.

4. Important Difference

Monitoring Method	What You Monitor	Use Case
Instance IP	Individual EC2 health	infrastructure monitoring

Monitoring Method	What You Monitor	Use Case
Load Balancer DNS	Service endpoint	uptime / availability
Target Group health	LB view of instance health	best operational view

💡 Best practice in AWS setups

Use **two layers**:

- 1 Monitor **Load Balancer DNS** → service availability
- 2 Monitor **target health via API** → instance health

If you want, I can also show a **very useful pattern**:

Automatically discover EC2 instances behind a Load Balancer and monitor them dynamically with Boto3 (no hard-coded IPs). This is what most monitoring systems do.

how can I subprocess into the current ec2 instance via dns name

Short answer: **you generally cannot “subprocess into” the current EC2 instance via a DNS name** the way you might be thinking. DNS names for load balancers resolve to the **load balancer**, not to the individual instance you are running on.

But depending on what you want to achieve, there are several workable patterns.

1. If you mean: run a shell command against the instance via DNS (SSH)

You can subprocess an SSH command if the DNS name points **directly to that EC2 instance**.

Example Python:

```
import subprocess
```

```
host = "ec2-12-34-56-78.compute-1.amazonaws.com"
```

```
cmd = [  
    "ssh",  
    "-i", "mykey.pem",  
    f"ec2-user@{host}",  
    "hostname"  
]  
  
result = subprocess.run(cmd, capture_output=True, text=True)  
print(result.stdout)
```

This works only if the DNS name resolves to **that instance**, not a load balancer.

2. If you're behind a Load Balancer (most likely case)

If you're using:

- **Application Load Balancer**
- **Network Load Balancer**

their DNS name **does not map to one specific instance**.

Example:

my-alb-123.eu-west-1.elb.amazonaws.com

Requests will go to **any healthy target**.

So you cannot reliably SSH or subprocess to **your current instance** via that DNS.

3. Correct Way: Get the Current Instance ID

Inside an EC2 instance you should use the **Instance Metadata Service**.

```
import requests  
  
instance_id = requests.get(  
    "http://169.254.169.254/latest/meta-data/instance-id"  
)  
.text  
  
print(instance_id)
```

This works without credentials and is the **standard AWS method**.

4. If you want the instance's DNS name

From metadata:

```
import requests
```

```
public_dns = requests.get(
    "http://169.254.169.254/latest/meta-data/public-hostname"
).text
```

```
print(public_dns)
```

Then you could use subprocess:

```
subprocess.run(["ssh", f"ec2-user@{public_dns}", "hostname"])
```

5. If your goal is “run commands on the current instance”

You **don't need SSH at all**. Just run the command locally:

```
import subprocess
```

```
subprocess.run(["hostname"])
```

6. Advanced / Best Practice: Use AWS Systems Manager

Instead of SSH, AWS recommends **AWS Systems Manager** Run Command.

Benefits:

- no SSH keys
 - works inside private VPC
 - secure audit logs
-

Summary

Goal	Solution
Run command on same instance	subprocess.run() locally

Goal	Solution
Get current instance info	EC2 metadata service
SSH via DNS	only if DNS points to instance
Behind load balancer	not possible via LB DNS

💡 If you want, tell me your **actual goal** (monitoring? service discovery? self-health checks?), and I can show a **much cleaner AWS-native pattern**. Most people solve this without DNS or SSH.

how about I get a load balancer and autoscaling, but use boto3 to subprocess into the running instance?

Yes, but **not the way “subprocess into the running instance” suggests**. With **autoscaling + load balancers**, instances are **ephemeral**, so the correct pattern is:

1. Discover the instance via AWS APIs
2. Execute a remote command via **SSM** (best practice) or **SSH**

Using **Boto3**, you typically combine:

- Auto Scaling → discover instances
- Elastic Load Balancing → find targets
- AWS Systems Manager → run commands

Best Practice: Run Commands via AWS SSM (No SSH Needed)

This works perfectly with autoscaling groups.

1. Discover instances in the Auto Scaling Group

```
import boto3
```

```
asg = boto3.client("autoscaling")
```

```
response = asg.describe_auto_scaling_groups(
    AutoScalingGroupNames=["my-asg"]
)

instances = [
    i["InstanceId"]
    for i in response["AutoScalingGroups"][0]["Instances"]
]

print(instances)
```

Example output:

```
['i-0123456789', 'i-987654321']
```

2. Execute a command on those instances

```
ssm = boto3.client("ssm")

response = ssm.send_command(
    InstanceIds=instances,
    DocumentName="AWS-RunShellScript",
    Parameters={
        "commands": ["hostname", "uptime"]
    }
)

print(response["Command"]["CommandId"])
```

This runs the command on **all running instances**.

If You Really Want to Use subprocess

Then you must:

1. Discover instance **public/private IP**
2. SSH to it

Example:

```

import boto3
import subprocess

ec2 = boto3.client("ec2")

response = ec2.describe_instances(
    InstanceIds=["i-0123456789"]
)

ip = response["Reservations"][0]["Instances"][0]["PublicIpAddress"]

subprocess.run([
    "ssh",
    f"ec2-user@{ip}",
    "hostname"
])

```

But this approach is fragile because autoscaling instances can:

- disappear anytime
- change IPs
- be replaced during scaling

If You're Using a Load Balancer

You can also discover instances from the **target group**.

```

elbv2 = boto3.client("elbv2")

targets = elbv2.describe_target_health(
    TargetGroupArn="your-target-group-arn"
)

instance_ids = [
    t["Target"]["Id"]
    for t in targets["TargetHealthDescriptions"]
]

```

Then send commands via SSM.

Real Architecture (Most Production Systems)

Monitoring script

| boto3



Auto Scaling Group



List EC2 instances



SSM Run Command



Execute command on instances

No SSH, no DNS, no IP tracking.

✅ Key takeaway

Autoscaling + load balancer environments should **never rely on fixed IPs or DNS to access instances**.

Instead use:

- **instance discovery via AWS APIs**
- **remote execution via SSM**

💡 If you'd like, I can also show a **very useful 40-line script** that:

- discovers the **load balancer → target group → instances**
- runs commands on them
- returns the output

This is extremely useful for **autoscaling debugging and monitoring**.

I like that method, but only need to do one instance this is active with the load balancer

If you only need **one instance that is currently active behind the load balancer**, the clean approach is:

1. Ask the load balancer which targets are **healthy**
2. Pick **one instance**
3. Run the command on that instance via **SSM**

This avoids dealing with IPs, SSH keys, or DNS randomness.

We'll use:

- Boto3
- Elastic Load Balancing
- AWS Systems Manager

Step 1 — Get a Healthy Instance From the Target Group

```
import boto3
```

```
elbv2 = boto3.client("elbv2")
```

```
target_group_arn = "YOUR_TARGET_GROUP_ARN"
```

```
response = elbv2.describe_target_health(  
    TargetGroupArn=target_group_arn  
)
```

```
healthy_instances = [  
    t["Target"]["Id"]  
    for t in response["TargetHealthDescriptions"]  
    if t["TargetHealth"]["State"] == "healthy"  
]
```

```
instance_id = healthy_instances[0]
```

```
print("Chosen instance:", instance_id)
```

This ensures you pick an **instance currently serving traffic**.

Step 2 — Run a Command on That Instance

```
ssm = boto3.client("ssm")

response = ssm.send_command(
    InstanceIds=[instance_id],
    DocumentName="AWS-RunShellScript",
    Parameters={
        "commands": ["hostname"]
    }
)

command_id = response["Command"]["CommandId"]

print("Command ID:", command_id)
```

Step 3 — Retrieve the Output

```
import time

time.sleep(2)

output = ssm.get_command_invocation(
    CommandId=command_id,
    InstanceId=instance_id
)

print(output["StandardOutputContent"])
```

Example result:

```
ip-10-0-12-45
```

Minimal Full Script

```
import boto3, time

elbv2 = boto3.client("elbv2")
ssm = boto3.client("ssm")

tg = "TARGET_GROUP_ARN"
```

```
targets = elbv2.describe_target_health(TargetGroupArn=tg)
```

```
instance = next(  
    t["Target"]["Id"]  
    for t in targets["TargetHealthDescriptions"]  
    if t["TargetHealth"]["State"] == "healthy"  
)
```

```
cmd = ssm.send_command(  
    InstanceIds=[instance],  
    DocumentName="AWS-RunShellScript",  
    Parameters={"commands": ["uptime"]}  
)
```

```
cmd_id = cmd["Command"]["CommandId"]
```

```
time.sleep(2)
```

```
result = ssm.get_command_invocation(  
    CommandId=cmd_id,  
    InstanceId=instance  
)
```

```
print(result["StandardOutputContent"])
```

✅ Why this works well with autoscaling

- always selects a **healthy instance**
- no IP tracking
- no SSH
- works even if instances are replaced

💡 **Small improvement many people add:** randomly choose a healthy instance instead of always picking the first one. That prevents hitting the same instance every time. I can show that pattern if you want.